

## CES - BOE 4.3 Reports module: steps and unit tests

What this adds: a Reports menu in CES. The user opens a report, enters parameters such as startDate and endDate, and the PDF from the Crystal report in BOE 4.3 is shown in the page. CES calls the TTC ReportService (Connect, GetReport, Disconnect). Output is PDF only. The service account, password, folder and other values are read from SBCES.CesSettings, one set per environment.

### What is in the zip

| Folder / file                                   | What it is                         | Where it goes in CES                       |
|-------------------------------------------------|------------------------------------|--------------------------------------------|
| sql\01_BOE_Reports_Setup.sql                    | Tables, settings, first report     | Run on each environment database           |
| sql\02_BOE_Reports_Rollback.sql                 | Undo script                        | Only if you need to remove the module      |
| code\Connected Services\BOEService\Reference.cs | WCF proxy generated from the WSDL  | CES project: Connected Services\BOEService |
| code\Controllers\Reports\ReportsController.cs   | Controller                         | CES project: Controllers\Reports           |
| code\Models\Reports\ReportModels.cs             | Model classes                      | CES project: Models\Reports                |
| code\Views\Reports\Index.cshtml, Viewer.cshtml  | Views (script is inline in Viewer) | CES project: Views\Reports                 |
| code\Snippets                                   | Menu item and csproj lines         | Copy into _Layout.cshtml and the .csproj   |
| tests\CES.BoeReports.Tests                      | xUnit test project                 | Add to the solution (optional)             |
| tools\generate-reference.ps1                    | Regenerates Reference.cs           | Only if the service contract changes       |

## Before you start

- Visual Studio 2022 or newer with the ASP.NET workload, and the .NET 8 SDK (the .NET 10 SDK also works).
- Get these values from the BOE Administrator, for DEV first and later for UAT and PROD: application user, its password, EnvironmentCode, and the BOE report folder. Also the exact report name and the exact Crystal prompt names (startDate, endDate).
- Work on a branch. Back up the DEV database or be ready to run the rollback script.
- Note the name of the CES web project: the .csproj that contains Controllers\Base\BaseController.cs.

## Step 1. Database

1. Open SQL Server Management Studio and connect to the DEV CES database.
2. Open sql\01\_BOE\_Reports\_Setup.sql.
3. Section 4: set @code, @title, @boe (the exact BOE report name) and @p1, @p2 (the exact Crystal prompt names, for example startDate and endDate).
4. Section 5: set @env, @user, @pwd, @folder and @envCode. The password is stored as plain text.
5. Run the script (F5). It is safe to run again.
6. Check: SELECT \* FROM SBCES.BoeReportMapping; SELECT \* FROM SBCES.BoeReportParameter; SELECT \* FROM SBCES.CesSettings WHERE Parameter LIKE 'BOE[\_]%. You should see 1 report, 2 parameters and 8 settings.
7. Repeat on UAT and PROD later, with that environment's own values.

## Step 2. Add the NuGet packages

The proxy needs two Microsoft packages: System.ServiceModel.Http and System.ServiceModel.Primitives, version 8.0.0 or a newer 8.x. No DLL is referenced by hand. Use one of the three ways below.

### Option A: Visual Studio

1. In Solution Explorer, right-click the CES web project, then Manage NuGet Packages.
2. Open the Browse tab. In the Package source list on the right, choose nuget.org (not All). This avoids the Telerik login prompt.
3. Search System.ServiceModel.Http. Choose the one by Microsoft. Set Version to 8.0.0, then Install, then accept the license.
4. Search System.ServiceModel.Primitives. Choose the one by Microsoft. Version 8.0.0, Install.
5. Open the Installed tab. Both packages should be listed. They also appear under Dependencies, then Packages, in Solution Explorer.

### Option B: command line

```
cd <folder that contains the CES web project .csproj>
dotnet add package System.ServiceModel.Http --version 8.0.0
dotnet add package System.ServiceModel.Primitives --version 8.0.0
dotnet restore
```

### Option C: edit the .csproj

Open the .csproj (double-click the project in Visual Studio) and add this block inside the Project element. The same lines are in code\Snippets\CES.csproj.snippet.xml. Save, then run dotnet restore.

```
<ItemGroup>
  <PackageReference Include="System.ServiceModel.Http" Version="8.0.0" />
  <PackageReference Include="System.ServiceModel.Primitives" Version="8.0.0" />
</ItemGroup>
```

Build pipelines: the Azure DevOps agent must be able to restore these two packages from nuget.org or from your internal feed. If the pipeline uses only an internal feed, ask the build team to allow them through the upstream source.

## Step 3. Add Reference.cs and reference it

Reference.cs is the proxy that svcutil generated from the ReportService WSDL. It holds the client class ReportServiceClient and the data classes (BOEConnection, BOEReportRequest, RptInputSingleValueParameter and others). Its namespace is BOEService. It is part of the CES project itself, so there is no DLL and no project reference to add.

1. In Windows Explorer, open the CES web project folder (the one with the .csproj).
2. Create the folder Connected Services, and inside it the folder BOEService.
3. Copy code\Connected Services\BOEService\Reference.cs from the zip into that folder. The result is <CES project>\Connected Services\BOEService\Reference.cs.
4. In Visual Studio, click Show All Files in Solution Explorer. If Reference.cs shows with a dotted outline, right-click it and choose Include In Project. Projects in the SDK format (the .NET 8 default) include every .cs file under the folder, so this is usually already done.
5. Select Reference.cs and check Properties: Build Action = C# compiler.
6. Do not edit the file and do not change its namespace. If you need extra members, put them in a separate file as a partial class.
7. Reference it from code with one line at the top of the controller: using BOEService;. ReportsController.cs already has this line.

- Build the project (Ctrl+Shift+B). Reference.cs must compile with 0 errors. Right-click ReportServiceClient in the controller and choose Go To Definition to confirm it opens Reference.cs.
- You do not need the dotnet-svcutil-params.json file, the NuGet.config from the generator folder, or the BoeProxy project. They were only used to create Reference.cs.

## Step 4. Add the model, controller and views

- Copy code\Models\Reports\ReportModels.cs to <CES project>\Models\Reports.
- Copy code\Controllers\Reports\ReportsController.cs to <CES project>\Controllers\Reports.
- Copy code\Views\Reports\Index.cshtml and Viewer.cshtml to <CES project>\Views\Reports. Create the folder if it is missing.
- There is no separate JavaScript file. The script is at the bottom of Viewer.cshtml.
- Match the namespaces to your project. The files use BCES.Controllers.Base, BCES.Data and BCES.Models.Reports. If your namespaces differ, change the using lines and the namespace lines in the three .cs files and the @using / @model lines in the views.
- Match the constructor. The controller calls base(dapper, httpContextAccessor) and dapper.CreateConnection(). Compare with another existing controller and copy its constructor pattern if it differs.
- Build again. Fix only namespace or constructor errors. Any error that mentions BOEService means Step 2 or 3 is not finished.

## Step 5. Add the menu item

- Open Views\Shared\\_Layout.cshtml.
- Open code\Snippets\\_Layout\_Reports\_menu.txt from the zip.
- Paste the Reports item inside the existing Kendo menu items, just before the Help item. If other menu items are shown only to some roles, wrap the Reports item in the same ViewBag.RoleId check.
- Reports are limited per role by BoeReportMapping.RequiredRoleId. Leave it empty to allow every logged-in role.

## Step 6. Check the settings, then run in DEV

| CesSettings Parameter       | Value                                                     | Notes                                                    |
|-----------------------------|-----------------------------------------------------------|----------------------------------------------------------|
| BOE_ServiceUrl              | https://boeservice.prod.int.ttc.ca/ReportService.svc/soap | Same for all environments. HTTPS only.                   |
| BOE_EnvironmentCode         | From the BOE Administrator                                | Chooses DEV, UAT or PROD inside the service.             |
| BOE_UserName / BOE_Password | BOE application account                                   | Password in plain text. Limit read access to this table. |
| BOE_PasswordEncrypted       | false                                                     | CES sends the password as plain text.                    |
| BOE_ReportFolder            | BOE report folder                                         | Used when a report has no folder of its own.             |
| BOE_Timeout                 | 60                                                        | Seconds. Raise it for slow reports.                      |
| BOE_EnableAudit             | true                                                      | Writes one row per run to BoeReportAudit.                |

- Fill in the REPLACE\_WITH values in CesSettings (UPDATE SBCES.CesSettings SET Value = '...' WHERE Parameter = '...').

2. Run CES in Visual Studio (F5) and sign in. Open Reports from the menu.
3. Open the report, pick startDate and endDate, and click Run Report. The PDF appears in the frame.

## Step 7. Deploy to UAT and PROD

1. Run the SQL on the UAT database, with the UAT user, password, folder and EnvironmentCode. Then the same for PROD.
2. Follow the normal RDM DevOps process for code: commit, build, and a Change Order for the environments after DEV.
3. The service URL stays the same in every environment. Only the CesSettings rows differ.
4. After deploy, run the manual tests T1, T4, T5 and T12 below in that environment.

## How to unit test

There are three layers. Use the first two while you build, and the manual list for each environment.

### Layer 1. Logic tests, no network

These test the parameter rules: date format, number format, string length. They need no database and no BOE.

1. Copy the tests folder into your solution folder, so you have <solution>\tests\CES.BoeReports.Tests next to <solution>\CES (the web project).
2. Open CES.BoeReports.Tests.csproj. The ProjectReference is ..\..\CES\CES.csproj. Change it to the real path and file name of the CES web project.
3. Add the project to the solution: right-click the solution, Add, Existing Project.
4. In the CES web project .csproj add the InternalsVisibleTo block from code\Snippets\CES.csproj.snippet.xml. This lets the tests call the internal method TryNormalize.
5. In ParameterNormalizeTests.cs change the two using lines if your namespaces differ from BCES.
6. Open Test, then Test Explorer, then Run All Tests. Or run: dotnet test tests\CES.BoeReports.Tests
7. Expected: all ParameterNormalizeTests pass (dates, numbers, strings). The smoke tests are skipped, which is normal.

### Layer 2. Service smoke tests, real BOE, no CES

These call Connect, GetReport and Disconnect directly. They prove that the account, EnvironmentCode, folder, report name and prompt names are right, before CES is involved. Use a DEV account only.

1. Open a PowerShell window. Set the variables for this window only (nothing is saved to disk):

```
$env:BOE_TEST_URL      = "https://boeservice.prod.int.ttc.ca/ReportService.svc/soap"
$env:BOE_TEST_USER     = "<DEV application user>"
$env:BOE_TEST_PASSWORD = "<its password>"
$env:BOE_TEST_ENV      = "<DEV EnvironmentCode>"
$env:BOE_TEST_FOLDER   = "<DEV report folder>"
$env:BOE_TEST_REPORT   = "<exact report name>"
# optional: BOE_TEST_START_NAME, BOE_TEST_END_NAME, BOE_TEST_START, BOE_TEST_END
dotnet test tests\CES.BoeReports.Tests --filter BoeServiceSmokeTests
```

- Connect\_returns\_a\_token\_and\_Disconnect\_succeeds: the account, password and EnvironmentCode work.
- GetReport\_returns\_a\_pdf: the folder, report name and prompt names work, and the bytes start with %PDF.
- GetReport\_with\_an\_unknown\_report\_name\_faults: the service reports a fault for a bad name.

- Connect\_with\_a\_wrong\_password\_faults: bad credentials are rejected, not ignored.
- If GetReport fails with a fault that mentions a parameter, the prompt name or the date format is wrong. Fix it here before touching CES.

### Layer 3. Manual tests in CES (run in each environment)

| #   | Test                                                                  | Expected result                                                 |
|-----|-----------------------------------------------------------------------|-----------------------------------------------------------------|
| T1  | Run the three SELECT checks from Step 1                               | 1+ report, its parameters, 8 BOE_ settings                      |
| T2  | Leave a REPLACE_WITH value in CesSettings and run a report            | Clear 'not configured' error, no crash                          |
| T3  | Sign in as a role that is not allowed                                 | Report is not in the list; opening its URL gives 403            |
| T4  | Open Reports, then the report                                         | Two date pickers (Start Date, End Date) and the Run button      |
| T5  | Pick a valid range and Run Report                                     | PDF shown in the frame, with data for that range                |
| T6  | Start Date later than End Date                                        | Message 'Start Date must be on or before End Date', no BOE call |
| T7  | Leave a required date empty                                           | Message that the field is required                              |
| T8  | Type /Reports/Run?code=<code>&p_startDate=abc in the address bar      | 400 'invalid value'                                             |
| T9  | Add &p_hack=1 to a valid Run URL                                      | Report runs. The extra value is ignored                         |
| T10 | Use a report code that does not exist                                 | 404 'Report not found'                                          |
| T11 | Set the report name in BoeReportMapping to a wrong name               | Message 'BOE rejected the request' with the BOE text            |
| T12 | SELECT TOP 5 * FROM SBCES.BoeReportAudit ORDER BY AuditId DESC        | One row per run: your UserId, parameters, status 200            |
| T13 | Set BOE_Timeout to 1 and run a slow report                            | Failure message, audit row with status 500, no hang             |
| T14 | Run a report that returns a PDF over 1 MB                             | PDF opens fully                                                 |
| T15 | After several runs, check the BOE sessions with the BOE Administrator | No growing pile of sessions (Disconnect works)                  |
| T16 | Search the CES logs and the audit table                               | No password and no session token anywhere                       |

A test is passed only when the result matches. Record the date, environment and tester for T1, T4, T5 and T12 on each deployment.